

SYNTETIKA ENGINE

Master Plan V2

Rebuild Arsitektur & Performa dari NOL

\f0b7 Audio Engine dengan Node Pooling

\f0b7 Svelte 5 Reactive UI Framework

\f0b7 Virtual Scrolling Pattern Grid

\f0b7 IndexedDB Persistence

\f0b7 Command Pattern Undo/Redo

\f0b7 WebGL2 VAO Caching

\f0b7 Seeded PRNG — Zero GC Hot Path

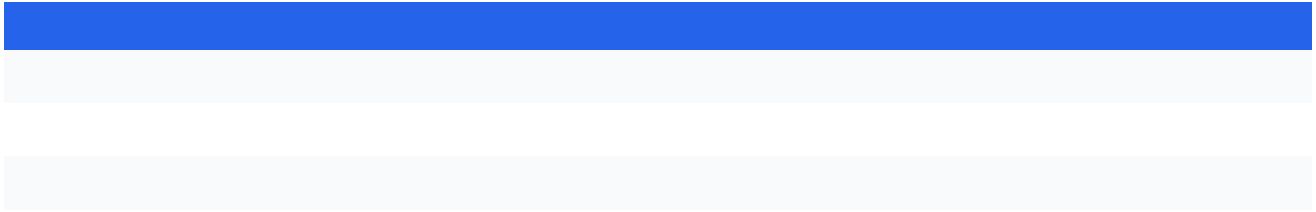
Daftar Isi

1. Analisis Masalah Arsitektur Saat Ini
2. Target: Stack Baru
3. Struktur Project Baru
4. Perbaikan Performa Kunci
5. Perbandingan Performa
6. Tabel Fitur Lengkap
7. Prioritas Migrasi
8. Audio Engine — Arsitektur Detail
9. UI Framework — Component Tree
10. State Management — Data Flow

1. Analisis Masalah Arsitektur Saat Ini

2. Target: Stack Baru

Rekomendasi utama: Svelte 5 + Vite + TypeScript. Alasan pemilihan:



Alternatif valid: Vue 3 + Composition API (ref/reactive). Lebih familiar di komunitas, bundle size ~16kB gzip.

3. Struktur Project Baru

Struktur direktori berbasis fitur (feature-based), bukan tipe file:

```
src/
  audio/
    engine.ts          # Web Audio API engine
    voices/            # AudioEngine class (synthesis, routing)
      drum.ts          # Per-voice synthesizer
      bass.ts          # Drum synthesis (kick, snare, hat, dll)
      melody.ts        # Bass voices (sub, acid, pluck)
      mono.ts          # Melody voices (bell, lead, pad)
      scheduler.ts     # Mono voices (moog, plucky, stabby, fm)
      pool.ts          # Lookahead sequencer scheduling
      noise-cache.ts   # AudioNode pool (reuse nodes)
  state/              # Noise buffer cache dengan LRU eviction
    store.ts          # Reactive state management
    pattern-store.ts  # Central state store (Svelte writable)
    persistence.ts    # Pattern matrix logic (bank/slot/step)
    undo.ts           # IndexedDB persistence (async, incremental)
  ui/                 # Command pattern undo/redo (delta-based)
    App.svelte        # Svelte 5 components
    Transport/        # Root layout
    Grid/             # Play/stop, BPM, tap tempo, nudge
    Mixer/            # Pattern grid (virtual scrolling)
    Controls/         # Volume faders
    Piano/            # Bank/slot, mode, sound style, loop
    Visual/           # MIDI keyboard overlay
    Matrix/           # Shader viewport + controls
    Panels/           # Pattern matrix (bank x slot)
  visual/            # Left/right collapsible panels
    shader-engine.ts  # WebGL shader system
    shader-editor/    # ISF shader renderer (WebGL2 + VAO)
    hydra-engine.ts   # Shader editing + gallery
    triggers.ts       # Hydra integration (single rAF)
  midi/              # Audio-reactive trigger decay
    manager.ts        # MIDI I/O
    mapping.ts        # Web MIDI API wrapper
    clock.ts          # Note-to-parameter mapping
  ai/                # MIDI clock sync
    engine.ts         # AI composition engine
    randomizer.ts     # AI orchestrator
    intent.ts         # Pattern generation
    ollama.ts         # NLP intent parser
  io/               # Ollama integration
    file.ts           # Input/Output
    resume.ts         # Project file import/export
    matrix-control.ts # OSC bridge untuk Resolume
  utils/            # BroadcastChannel sync
    prng.ts           # Utilities
    note-utils.ts     # Seeded PRNG (ganti Math.random)
    math.ts           # Note <-> frequency conversion
                    # Clamp, lerp, dll
```

4. Perbaikan Performa Kunci

4.1 AudioNode Pooling

Masalah sekarang: ~400 node/detik dibuat, tidak pernah di-disconnect(). Setiap OscillatorNode/GainNode/BiquadFilterNode baru adalah heavy object. Tanpa disconnect(), sub-graph tetap referenced oleh channel gain — memory leak.

Solusi: Pool yang me-reuse AudioNode objects:

```
class AudioNodePool {
  private pool: OscillatorNode[] = [];
  acquire(type: string, freq: number) {
    let osc = this.pool.pop();
    if (!osc) osc = ctx.createOscillator();
    osc.type = type;
    osc.frequency.value = freq;
    return osc;
  }
  release(osc) {
    osc.disconnect(); // CRITICAL
    try { osc.stop(); } catch {}
    if (pool.length < MAX) pool.push(osc);
  }
}
```

Dampak: Zero allocation di hot path. Eliminasi GC pressure dari node creation/disposal.

4.2 Seeded PRNG — Ganti Math.random()

Masalah: Math.random() dipanggil 248x/detik untuk complex drum (clickCluster, playNoiseDrum, dll). Setiap call mengalokasi memori internal untuk entropy collection.

Solusi: Mulberry32 PRNG — sangat cepat, zero allocation, deterministic:

```
function createPRNG(seed: number) {
  let s = seed | 0;
  return () => {
    s = (s + 0x602B79F5) | 0;
    let t = Math.imul(s ^ (s >> 15), 1 | s);
    t = (t + Math.imul(t ^ (t >> 7), 61 | t)) ^ t;
    return ((t ^ (t >> 14)) >> 0) / 4294967296;
  };
}
```

Dampak: Lebih cepat dari Math.random(), zero GC, deterministic untuk reproducible pattern.

4.3 Virtual Scrolling untuk Pattern Grid

Masalah: renderGrid() rebuild seluruh DOM (~160+ elemen) tiap ganti page/mode. Ini melibatkan createElement, innerHTML, classList toggle untuk setiap cell.

Solusi: Hanya render 16 step yang visible + 1 buffer row. Svelte 5 reactive blocks untuk update efisien:

```
{#each visibleSteps as step {step.index}}
  <StepCell {active} on:pointerdown={() => toggle(step.index)} />
{/each}
```

Dampak: DOM operations O(16) bukan O(160). Svelte compile-time reactivity menghilangkan applyStateToUI().

4.4 IndexedDB Persistence

Masalah: JSON.stringify(state) ~500KB tiap save, localStorage synchronous. Setiap pattern edit memicu full serialization.

Solusi: IndexedDB dengan incremental saves. Pattern data disimpan terpisah dari UI state, dengan compression:

```
const SAVE_KEYS = ['memory', 'presetLoopLengths', 'presetTrackRates', ...];
async function saveState(state: State) {
  const tx = db.transaction(['patterns', 'meta'], 'readwrite');
  await tx.store('patterns').put(compressPatterns(state.memory));
}
```

```
    await tx.store('meta').put(serializeMeta(state));  
  }  
}
```

Dampak: Async, non-blocking, compression mengurangi ukuran 60-80%. Tidak ada main thread blocking.

4.5 Command Pattern untuk Undo/Redo

Masalah: `JSON.parse(JSON.stringify(state))` untuk snapshot undo. 30 snapshot x 500KB = 15MB memory. Full clone tiap operasi.

Solusi: Simpan hanya inverse operation — dari 500KB menjadi ~32 bytes per command:

```
class ToggleStepCommand implements Command {  
  constructor(kind, bank, slot, track, step, prevValue) {}  
  apply(state) { state.memory[kind][bank][slot][track][step] = !prevValue; }  
  inverse() { return new ToggleStepCommand(..., !prevValue); }  
}
```

Dampak: Undo snapshot 500KB -> 32 bytes. Memory usage 15MB -> ~1KB.

4.6 WebGL2 dengan VAO Caching

Masalah: `gl.getAttribLocation("a_position")` dipanggil tiap frame — synchronous round-trip ke GPU driver. Set uniform di-loop untuk semua inputDefs meski tidak dipakai shader.

Solusi: Cache attribute location setelah program link. Gunakan WebGL2 Vertex Array Objects (VAO) untuk state caching.

```
compileShader(source) {  
  // compile, link ...  
  this._posLoc = gl.getAttribLocation(this.program, 'a_position');  
  // VAO — set attrib pointers sekali saja  
  this._vao = gl.createVertexArray();  
  gl.bindVertexArray(this._vao);  
  gl.enableVertexAttribArray(this._posLoc);  
  gl.vertexAttribPointer(this._posLoc, 2, gl.FLOAT, false, 0, 0);  
  gl.bindVertexArray(null);  
}
```

Dampak: Eliminasi synchronous GL round-trip tiap frame. VAO mengurangi driver overhead.

4.7 Single Unified rAF Loop

Masalah: Sekarang ada 2+ rAF chains — ShaderEngine punya sendiri, HydraEngine punya autoLoop sendiri. Mereka kompetisi main thread.

Solusi: Satu rAF loop untuk semua — audio scheduling, visual rendering, Hydra tick:

```
function renderLoop(time: number) {  
  scheduler.tick(time);           // audio lookahead  
  shaderEngine.render(time);      // ISF or passthrough  
  hydraEngine?.tick(time);        // Hydra manual advance (autoLoop=false)  
  requestAnimationFrame(renderLoop);  
}
```

Dampak: Satu source of truth untuk timing. Tidak ada kompetisi antar rAF. FPS limiting 30fps default.

[illegible]

7. Prioritas Migrasi

Phase 1: Core Engine

- Minggu 1-2
- \bullet audio/engine.ts + audio/pool.ts + audio/voices/*
 - \bullet audio/scheduler.ts (lookahead scheduling)
 - \bullet state/store.ts + state/pattern-store.ts
 - \bullet utils/prng.ts

Phase 2: UI Framework

- Minggu 2-4
- \bullet Setup Svelte + Vite + TypeScript project
 - \bullet App.svelte (layout, panels)
 - \bullet Grid/ components (virtual scroll)
 - \bullet Transport/, Controls/, Mixer/

Phase 3: Visual Engine

- Minggu 3-5
- \bullet visual/shader-engine.ts (WebGL2, VAO)
 - \bullet visual/hydra-engine.ts (single rAF)
 - \bullet visual/triggers.ts (decay system)

Phase 4: Integrasi

- Minggu 5-6
- \bullet MIDI, AI, Resolume, file I/O
 - \bullet state/persistence.ts (IndexedDB)
 - \bullet state/undo.ts (command pattern)

Phase 5: Polish & Test

- Minggu 6-8
- \bullet Testing, profiling, optimasi
 - \bullet Shader editor gallery
 - \bullet Matrix control, popup visual

8. Audio Engine — Arsitektur Detail

8.1 Audio Graph Baru

Signal flow dengan AudioNode pooling dan voice management:

```
[Pool] --acquire--> OscillatorNode / GainNode / BiquadFilterNode
      |
      +--> Voice: connect() -> channelGain -> master -> analyser -> destination
      |
      +--> release: disconnect() + stop() + return to pool

voice types:
- DrumVoice: fire-and-forget, pooled, no active tracking
- NoteVoice: monophonic per-kind, tracked via _activeVoices[kind]
- noteOn: acquire from pool, connect, schedule envelope
- retieNote: update frequency only (no node recreation)
- releaseNote: cancel envelope, disconnect, return to pool
```

8.2 Lookahead Scheduler

Scheduler dengan single RAF loop, lookahead window 500ms:

```
renderLoop(time) {
  const now = audioCtx.currentTime;
  const deadline = now + LOOKAHEAD_SEC; // 0.5s

  for each kind in [drum, bass, melody, other]:
    while nextStepTime[kind] < deadline:
      emit { kind, step, time: nextStepTime[kind] + swing }
      advance step (wrap at loopLength)
      nextStepTime[kind] += stepDuration / rate

  onTick(events); // schedule audio + MIDI + visual triggers
}
```

8.3 Node Pool Strategy

Pool size limits per node type:

Node lifecycle: acquire -> configure -> connect -> schedule stop -> disconnect -> release.

Kunci: OscillatorNode.stop() tidak otomatis disconnect dari graph. Disconnect() WAJIB dipanggil sebelum return ke pool untuk mencegah memory leak.

9. UI Framework — Component Tree

Struktur komponen Svelte 5:

```
App.svelte
+-- Transport.svelte      [play/stop, BPM, tap tempo, nudge, resync]
+-- GridSection.svelte
|   +-- GridToolbar.svelte [page nav, loop length, follow, clear, copy/paste]
|   +-- DrumGrid.svelte   [virtual scroll – hanya 16 visible steps]
|   |   +-- DrumRow.svelte x 7 (kick, snare, clap, ...)
|   |   +-- StepCell.svelte x 16 (visible)
|   +-- NoteGrid.svelte   [bass/melody/other]
|   |   +-- NoteRow.svelte x 4 (16, 32, 64, edit)
|   |   +-- StepCell.svelte x 16 (visible)
+-- MixerPanel.svelte     [4 faders + sound style picker]
+-- ControlPanel.svelte   [bank/slot, mode, sound style, randomize]
+-- VisualPanel.svelte
|   +-- ShaderViewport.svelte [canvas + overlay]
|   +-- ShaderControls.svelte [dual-range, color, vec2]
|   +-- VisualPresets.svelte  [save/load per slot]
+-- MatrixPanel.svelte     [8x8 bank x slot grid]
+-- MidiKeyboard.svelte    [piano overlay]
+-- SidePanel.svelte       [left: MIDI/Resolume/Visual; right: AI Chat]
```

Prinsip UI: Tidak ada innerHTML. Semua rendering via Svelte template. Update otomatis via reactive \$state.

10. State Management — Data Flow

Data flow baru dengan Svelte 5 \$state dan command-based undo:

```
// --- Store ---
let bpm = $state(120);
let mode = $state('drum');
let activeBanks = $state({ drum:0, bass:0, melody:0, other:0 });
let activeSlots = $state({ drum:0, bass:0, melody:0, other:0 });
let memory = $state(createEmptyMemory());

// --- Command (untuk undo/redo log) ---
class ToggleStepCmd {
  constructor(public kind, public bank, public slot, public track, public step, public prevvalue) {}
  apply() { memory[kind][bank][slot][track][step] = !prevvalue; }
  inverse() { return new ToggleStepCmd(kind, bank, slot, track, step, !prevvalue); }
}

// --- Undo stack (max 100 commands) ---
let undoStack = $state([]); // array of Command[]
let redoStack = $state([]);

function execute(cmd: Command) {
  cmd.apply();
  undoStack.push(cmd);
  redoStack = [];
  if (undoStack.length > 100) undoStack.shift();
  persistDebounce(); // IndexedDB — 350ms debounce
}

function undo() {
  const cmd = undoStack.pop();
  if (!cmd) return;
  cmd.inverse().apply();
  redoStack.push(cmd);
  persistDebounce();
}

// --- Persistence ---
const persistDebounce = debounce(350, async () => {
  await saveToIndexedDB({ bpm, mode, memory, ... });
});

// --- UI reactivity (automatic via $state) ---
$effect(() => {
  // Svelte tracks dependencies automatically
  // Any UI that reads $state() re-renders when it changes
  console.log('BPM changed to', bpm);
});
```

Prinsip state management:

- Semua state di Svelte \$state() — reactive, no manual applyStateToUI()
- Undo via Command pattern — simpan inverse operation, bukan full snapshot
- Persistence via IndexedDB — async, debounced 350ms, hanya simpan pattern yang berubah
- Broadcast ke matrix control/monitor via \$effect() — otomatis, bukan manual sync()

Siap Eksekusi

Master Plan V2 — Syntetika Engine

Dokumen ini adalah acuan untuk rebuild aplikasi dari NOL dengan arsitektur yang lebih efisien dan ringan.